

咖啡台 LED 控制 · TCP 自定义协议 v1.0

项	值
适用固件	coffe_mcu (ESP32-C5-N16R8)
版本	v1.0 (2026-09-15)
用途	客户/上位机作为 TCP 客户端主动连接设备，通过自定义 JSON 协议控制 10 路 LED，设备执行完毕后回一个应答报文。
说明	本协议用于对外控制；另保留标准 Modbus TCP (端口 502) 作为调试/兼容接口（见文末附录）。

1. 传输与连接

项	值
协议	TCP (面向连接)
监听端口	9001
连接方向	客户 (客户端) 主动 connect → 设备 (服务端) 应答；连接后由客户发命令
并发连接	最多 4 个 (多余连接会被拒绝)
心跳/保活	客户可用 ping 命令测试连接；连接超时可由客户自行决定重连
帧格式	每条命令一个 JSON 对象 + 一个换行符 \n；设备每条应答同样是一个 JSON 对象 + \n

⚠ 重要： 每条命令必须是单行 JSON (不带内部换行)，并以 \n (LF, 0x0A) 结尾。设备按 \n 切分消息。若发送多字节/含换行的美化 JSON，请自行压缩为单行。

2. 消息通用结构

2.1 请求 (客户 → 设备)

```
{ "seq": 123, "cmd": "led_set", "ch": 1, "on": true }
```

字段	类型	必填	说明
seq	int	否	客户自定序号；应答原样回传，用于请求-应答匹配
cmd	string	是	命令名 (见第 4 节)

其余	-	按命令	各命令参数
----	---	-----	-------

2.2 应答（设备 → 客户，执行完毕后发送）

成功：

```
{ "seq": 123, "ok": true, "cmd": "led_set", "data": { "ch": 1, "on": true } }
```

失败：

```
{ "seq": 123, "ok": false, "cmd": "led_set", "err": "INVALID_CHANNEL", "msg": "ch 需为 1..10" }
```

字段	类型	说明
seq	int?	若请求带了 seq 则回传，否则无此字段
ok	bool	true 成功 / false 失败
cmd	string	对应命令名（无法解析时可能为 ""）
data	object?	成功时的业务数据，见各命令
err	string?	失败时的错误码，见第 5 节
msg	string?	失败时的中文/说明文案

3. LED 通道定义

- 共 10 路：通道号 CH1 ~ CH10（取值 1..10）
- 每个通道状态：on (true / 1) = 亮，off (false / 0) = 灭
- 与 Modbus 寄存器对应关系见附录（Modbus 地址 0x0000..0x0009 = CH1..CH10）

4. 命令集

4.1 ping — 连接测试 / 心跳

请求：{ "cmd": "ping" }

成功应答 data: { "pong": true }

4.2 led_set — 单路开/关

请求：{ "cmd": "led_set", "ch": 1, "on": true }

成功应答 data: { "ch": 1, "on": true }

错误：INVALID_CHANNEL（ch 不在 1..10）、INVALID_VALUE（on 非布尔）

4.3 led_set_all — 全开 / 全关

请求: { "cmd": "led_set_all", "on": true }

成功应答 data: { "count": 10, "on": true }

错误: INVALID_VALUE (on 非布尔)

4.4 led_set_batch — 多条一起设置（一次应答）

请求: { "cmd": "led_set_batch", "list": [{"ch":1,"on":true}, {"ch":3,"on":false}] }

成功应答 data: { "applied": 2 }

说明: 先整体校验再统一执行; 任一条非法则整条返回错误、不执行任何一条。applied = 成功设置的条数。

错误: INVALID_CHANNEL、INVALID_VALUE、INVALID_LIST (list 缺失/非数组)

4.5 led_status — 读当前全部状态

请求: { "cmd": "led_status" }

成功应答 data:

```
{ "channels": [ {"ch":1,"on":true}, {"ch":2,"on":false}, ... ] }
```

5. 错误码

err	说明
INVALID_JSON	JSON 无法解析 / 空串
INVALID_CMD	cmd 缺失或未知
INVALID_CHANNEL	ch 不在 1..10
INVALID_VALUE	on 不是布尔、或数值非法
INVALID_LIST	list 缺失或不是数组 / 数组元素格式非法
INTERNAL	设备内部异常 (如 NVS 写失败)

6. 交互示例

客户 connect 192.168.1.100:9001, 依次发送 (每行为一个请求):

点亮 CH3

```
{"seq":1,"cmd":"led_set","ch":3,"on":true}
```

设备执行后应答:

```
{"seq":1,"ok":true,"cmd":"led_set","data":{"ch":3,"on":true}}
```

读全部状态

请求: {"cmd":"led_status"}

应答示例：

```
{"ok":true,"cmd":"led_status","data":{"channels":[{"ch":1,"on":true},{"ch":2,"on":false}, {"ch":3,"on":true}, {"ch":4,"on":false}, {"ch":5,"on":false}, {"ch":6,"on":false}, {"ch":7,"on":false}, {"ch":8,"on":false}, {"ch":9,"on":false}, {"ch":10,"on":false}]}}
```

非法通道

请求：{"seq":2,"cmd":"led_set","ch":11,"on":true}

应答：

```
{"seq":2,"ok":false,"cmd":"led_set","err":"INVALID_CHANNEL","msg":"ch 需为 1..10"}
```

7. 安全说明

- 本协议当前无鉴权/令牌：连上即可控制全部 LED。请部署于可信局域网 / VLAN。
- 后续如需，可加访问令牌（连接时或每条命令校验），见后续版本 v1.1。

附录 A · 与 Modbus TCP 从站（端口 502，调试用）

设备同时在 端口 502 提供标准 Modbus TCP 从站（用于调试/兼容）：

Modbus 项	值
功能码	0x03 读保持寄存器 / 0x06 写单个 / 0x10 写多个
寄存器地址	0x0000..0x0009 = CH1..CH10
寄存器值	0 =灭, 1 =亮
工具	mbpoll / Modbus Poll / pymodbus

注：Modbus 从站与自定义 TCP 协议（9001）可同时监听，互不干扰，都只在网络层占用端口、不占串口。